

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление 02.03.01 Математика и Компьютерные науки

Курсовая работа
по дисциплине «Анализ больших данных »

Студент: _____ Карпович Лидия Александровна

Преподаватель: _____ Константинов Андрей Владимирович

«____» _____ 20__ г.

Содержание

1	Введение	3
2	Архитектура системы	3
2.1	Компоненты	3
2.2	Схема взаимодействия	4
3	Инфраструктура: docker-compose	4
4	База данных ClickHouse	5
4.1	Схема таблиц	5
5	Стриминговый генератор данных	6
6	Apache Spark: распределённый кластер	7
6.1	Конфигурация кластера	7
6.2	Генерация датасета (generate_data.py)	8
6.3	Обучение ML модели (distributed_ml.py)	9
7	Apache Airflow: оркестрация	10
7.1	DAG пайплайна	10
7.2	Граф задач	11
8	Запись результатов в ClickHouse	11
9	Визуализация: Grafana	12
9.1	Дашборд	12
10	Распределённость Spark: как работает параллельно	13
10.1	Что происходит на каждом узле	13
10.2	Поведение при отказе узла	14
11	Результаты	14
11.1	Запуск нескольких итераций	14
11.2	Итоговые метрики	14
12	Выводы	14

1 Введение

В данной работе реализован end-to-end пайплайн обработки больших данных, включающий:

- непрерывную генерацию и запись событий в аналитическую базу данных ClickHouse;
- распределённую генерацию датасета и обучение модели машинного обучения на кластере Apache Spark (1 мастер + 3 воркера);
- оркестрацию всех задач через Apache Airflow с механизмом повторных попыток;
- визуализацию результатов в Grafana.

Все компоненты развёрнуты в Docker-контейнерах и управляются через `docker-compose`.

2 Архитектура системы

2.1 Компоненты

Система состоит из следующих сервисов:

Таблица 1: Сервисы системы

Сервис	Образ / Порт	Назначение
clickhouse	clickhouse-server:24.3 / 8123, 9000	Аналитическая СУБД, хранит события и результаты ML
generator	custom build	Непрерывная генерация событий → ClickHouse
airflow	custom build / 8080	Оркестратор задач, DAG пайплайна
spark-master	custom build / 7077, 8081	Мастер Spark кластера
spark-worker-1/2/3	custom build	Три воркера для параллельной обработки
grafana	grafana:10.4.0 / 3000	Дашборды и мониторинг

2.2 Схема взаимодействия

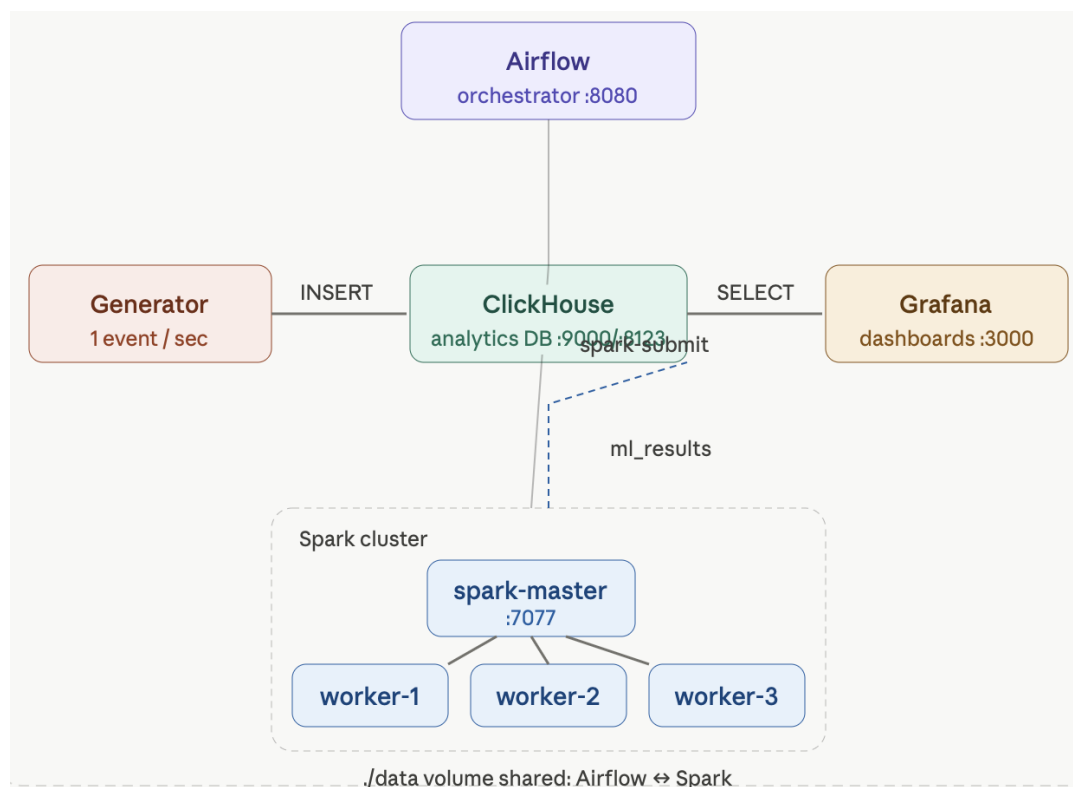


Рис. 1: Архитектура системы

Поток данных:

1. **Generator** каждую секунду генерирует случайные признаки и записывает в `analytics.events`.
2. **Airflow DAG** запускается вручную и последовательно выполняет четыре задачи.
3. `generate_data` — Spark генерирует 3 000 000 строк и сохраняет в Parquet.
4. `distributed_ml` — Spark обучает логистическую регрессию на кластере.
5. `write_clickhouse` — результаты (ассигасу, кол-во строк) записываются в `analytics.ml_results`.
6. **Grafana** читает из ClickHouse и отображает метрики в реальном времени.

3 Инфраструктура: docker-compose

Все сервисы описаны в файле `docker-compose.yml`. Ключевые моменты:

- **ClickHouse** стартует первым; остальные сервисы ждут `healthcheck` (SELECT 1 каждые 5 секунд, 10 попыток).
- **Generator** имеет `restart: always` — перезапускается при любом падении.

- **Airflow** монтирует `/var/run/docker.sock` для запуска `docker exec` внутри BashOperator.
- **Shared volume** `./data:/opt/airflow/data` примонтирован ко всем Spark-контейнерам и Airflow — обеспечивает обмен Parquet-файлами.

Листинг 1: Фрагмент `docker-compose.yml`: healthcheck и зависимости

```
clickhouse:
  image: clickhouse/clickhouse-server:24.3
  healthcheck:
    test: ["CMD", "clickhouse-client", "--query", "SELECT 1"]
    interval: 5s
    retries: 10

generator:
  build: ./generator
  depends_on:
    clickhouse:
      condition: service_healthy
  restart: always
```

4 База данных ClickHouse

4.1 Схема таблиц

В базе `analytics` созданы две таблицы:

Листинг 2: `init.sql` — схема таблиц ClickHouse

```
CREATE DATABASE IF NOT EXISTS analytics;

—
CREATE TABLE IF NOT EXISTS analytics.events (
  id      UInt64,
  x1      Float64,
  x2      Float64,
  x3      Float64,
  label   Float64
)
ENGINE = MergeTree()
ORDER BY id;

—
CREATE TABLE IF NOT EXISTS analytics.ml_results (
  ts          DateTime,
  accuracy    Float64,
  rows_processed UInt64
)
ENGINE = MergeTree()
ORDER BY ts;
```

Движок **MergeTree** — основной движок ClickHouse для аналитических нагрузок, поддерживает эффективную вставку и агрегацию большого объёма данных.

5 Стриминговый генератор данных

Сервис **generator** непрерывно генерирует синтетические события и записывает их в ClickHouse.

Листинг 3: generator.py — генерация и запись событий

```
from clickhouse_driver import Client
import random
import time

def connect():
    while True:
        try:
            client = Client(
                host="clickhouse",
                database="analytics",
                user="admin",
                password="admin123"
            )
            client.execute("SELECT 1")
            print("Connected")
            return client
        except Exception as e:
            print("Waiting_ClickHouse", e)
            time.sleep(5)

client = connect()

while True:
    feature1 = random.uniform(0, 100)
    feature2 = random.uniform(0, 50)
    feature3 = random.uniform(0, 30)
    label = 1.0 if (feature1 + feature2 > 75) else 0.0

    row = [(
        random.randint(1, 1000),
        float(feature1),
        float(feature2),
        float(feature3),
        label
    )]
    client.execute(
        "INSERT INTO analytics.events_(id, _x1, _x2, _x3, _label) VALUES",
        row
    )
    print("Inserted", row)
    time.sleep(1)
```

Логика генерации:

- Три числовых признака $x_1 \in [0, 100]$, $x_2 \in [0, 50]$, $x_3 \in [0, 30]$.
- Метка `label` = 1 если $x_1 + x_2 > 75$, иначе 0 — детерминированная зависимость для демонстрации обучаемости модели.
- Вставка каждую секунду через нативный протокол ClickHouse (порт 9000).
- При недоступности ClickHouse — повторные попытки подключения каждые 5 секунд.

6 Apache Spark: распределённый кластер

6.1 Конфигурация кластера

Кластер состоит из одного мастера и трёх воркеров, все собираются из единого Dockerfile:

Листинг 4: spark/Dockerfile

```
FROM apache/spark:3.5.0
USER root
RUN pip install --no-cache-dir \
    numpy \
    pandas \
    scikit-learn \
    clickhouse-driver \
    pyarrow
```

Мастер принимает задачи на порту 7077. Воркеры регистрируются у мастера автоматически при старте. Все Spark-контейнеры монтируют общий volume `./data:/opt/airflow/data`, что позволяет обмениваться Parquet-файлами между задачами.

The screenshot shows the Spark Master Web UI for Spark 3.5.0. The URL is `spark://spark-master:7077`. The status is ALIVE. There are 2 alive workers, 20 total cores in use, and 13.5 GiB total memory in use. There is 1 running application and 1 completed application. The workers table shows one worker in a DEAD state and two in ALIVE state. The running applications table shows one application in a RUNNING state. The completed applications table shows one application in a FINISH state.

Worker Id	Address	State	Cores	Memory	Resources
worker-20260521195034-172.19.0.4-36435	172.19.0.4:36435	DEAD	10 (10 Used)	6.8 GiB (1024.0 MiB Used)	
worker-20260521195034-172.19.0.5-38343	172.19.0.5:38343	ALIVE	10 (10 Used)	6.8 GiB (1024.0 MiB Used)	
worker-20260521195034-172.19.0.6-36923	172.19.0.6:36923	ALIVE	10 (10 Used)	6.8 GiB (1024.0 MiB Used)	

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260521195256-0001	(kill) FaultToleranceDemo	20	1024.0 MiB		2026/05/21 19:52:56	root	RUNNING	14 s

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260521195115-0000	FaultToleranceDemo	30	1024.0 MiB		2026/05/21 19:51:15	root	FINISH	

Рис. 2: Spark Master Web UI — демонстрация "убит 1 узел"

URL: spark://spark-master:7077
Alive Workers: 3
Cores in use: 30 Total, 0 Used
Memory in use: 20.3 GiB Total, 0.0 B Used
Resources in use:
Applications: 0 Running, 8 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

Workers (3)

Worker Id	Address	State	Cores	Memory	Resources
worker-20260524132036-172.19.0.4-41091	172.19.0.4:41091	ALIVE	10 (0 Used)	6.8 GiB (0.0 B Used)	
worker-20260524132036-172.19.0.5-35671	172.19.0.5:35671	ALIVE	10 (0 Used)	6.8 GiB (0.0 B Used)	
worker-20260524132036-172.19.0.6-33109	172.19.0.6:33109	ALIVE	10 (0 Used)	6.8 GiB (0.0 B Used)	

Running Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Completed Applications (8)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260524132424-0007	DistributedML	30	1024.0 MiB		2026/05/24 13:24:24	root	FINISHED	36 s
app-20260524132345-0006	DistributedML	30	1024.0 MiB		2026/05/24 13:23:45	root	FINISHED	33 s
app-20260524132307-0005	DistributedML	30	1024.0 MiB		2026/05/24 13:23:07	root	FINISHED	32 s
app-20260524132226-0004	DistributedML	30	1024.0 MiB		2026/05/24 13:22:26	root	FINISHED	35 s
app-20260524132210-0003	GenerateData	30	1024.0 MiB		2026/05/24 13:22:10	root	FINISHED	10 s
app-20260524132157-0002	GenerateData	30	1024.0 MiB		2026/05/24 13:21:57	root	FINISHED	9 s
app-20260524132143-0001	GenerateData	30	1024.0 MiB		2026/05/24 13:21:43	root	FINISHED	10 s
app-20260524132126-0000	GenerateData	30	1024.0 MiB		2026/05/24 13:21:26	root	FINISHED	13 s

Рис. 3: Spark Master Web UI — кластер из 3 воркеров

6.2 Генерация датасета (generate_data.py)

Листинг 5: generate_data.py — генерация 3 млн строк на Spark

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import rand, col

spark = SparkSession.builder \
    .appName("GenerateData") \
    .getOrCreate()

df = spark.range(0, 3000000)
df = df \
    .withColumn("x1", rand()) \
    .withColumn("x2", rand()) \
    .withColumn("x3", rand())

df = df.withColumn(
    "label",
    (col("x1") + col("x2") > 1).cast("int")
)

df = df.repartition(12)
df.write.mode("overwrite").parquet("/opt/airflow/data/ml_input")
spark.stop()

```

Как работает параллельно:

- `spark.range(0, 3 000 000)` создаёт RDD из 3 миллионов строк, автоматически разбитый на партии.
- `repartition(12)` явно разбивает данные на 12 партий — по 4 на каждый из 3 воркеров.
- Запись в Parquet происходит параллельно: каждый воркер пишет свои партии (`part-00000-*.parquet, ..., part-00011-*.parquet`).

6.3 Обучение ML модели (distributed_ml.py)

Листинг 6: distributed_ml.py — обучение логистической регрессии

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator

spark = SparkSession.builder \
    .appName("DistributedML") \
    .getOrCreate()

df = spark.read.parquet("/opt/airflow/data/ml_input")

assembler = VectorAssembler(
    inputCols=["x1", "x2", "x3"],
    outputCol="features"
)
df = assembler.transform(df)

train, test = df.randomSplit([0.8, 0.2], seed=42)

lr = LogisticRegression(maxIter=10)
model = lr.fit(train)
pred = model.transform(test)

evaluator = BinaryClassificationEvaluator(
    labelCol="label",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"
)
metric = evaluator.evaluate(pred)
rows = df.count()

spark.createDataFrame(
    [(float(metric), int(rows))],
    ["metric", "rows_processed"]
).write.mode("overwrite").parquet("/opt/airflow/data/ml_result")

spark.stop()
```

Этапы обработки:

1. Чтение Parquet — Spark распределяет партии по воркерам.
2. `VectorAssembler` объединяет `x1`, `x2`, `x3` в вектор признаков.
3. Разбивка 80/20 на train/test.
4. `LogisticRegression` обучается распределённо: градиентный спуск выполняется на всех воркерах параллельно.
5. Метрика **AUC-ROC** вычисляется на тестовой выборке.

6. Результат записывается в Parquet.

Метка `label = 1 if (x1 + x2 > 1)` является линейно разделимой, поэтому логистическая регрессия достигает $AUC \approx 0.9998$.

7 Apache Airflow: оркестрация

7.1 DAG пайплайна

Листинг 7: `spark_ml_pipeline.py` — DAG с retry

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

default_args = {
    "retries": 3,
    "retry_delay": timedelta(seconds=30),
}

with DAG(
    dag_id="spark_ml_pipeline",
    start_date=datetime(2025, 1, 1),
    schedule=None,
    catchup=False,
    default_args=default_args,
):
    generate = BashOperator(
        task_id="generate_data",
        bash_command="""
docker_exec_spark-master \
  /opt/spark/bin/spark-submit \
  --master_spark://spark-master:7077 \
  /opt/spark/work-dir/generate_data.py
"""

    ml = BashOperator(
        task_id="distributed_ml",
        bash_command="""
docker_exec_spark-master \
  /opt/spark/bin/spark-submit \
  --master_spark://spark-master:7077 \
  /opt/spark/work-dir/distributed_ml.py
"""

    clickhouse = BashOperator(
        task_id="write_clickhouse",
        bash_command="""
docker_exec_spark-master \
  /opt/spark/bin/spark-submit \
  --master_spark://spark-master:7077 \
  /opt/spark/work-dir/write_clickhouse.py
"""
```

```

    cc/opt/spark/work-dir/write_clickhouse.py
    """ )

    validate = BashOperator(
        task_id="validate",
        bash_command="echo_Pipeline_finished"
    )

    generate >> ml >> clickhouse >> validate

```

7.2 Граф задач

Задачи выполняются строго последовательно:

generate_data → distributed_ml → write_clickhouse → validate

Механизм retry: при падении любой задачи Airflow ждёт 30 секунд и повторяет попытку, до 3 раз. В UI упавшая задача отображается оранжевым цветом со статусом `up_for_retry`.

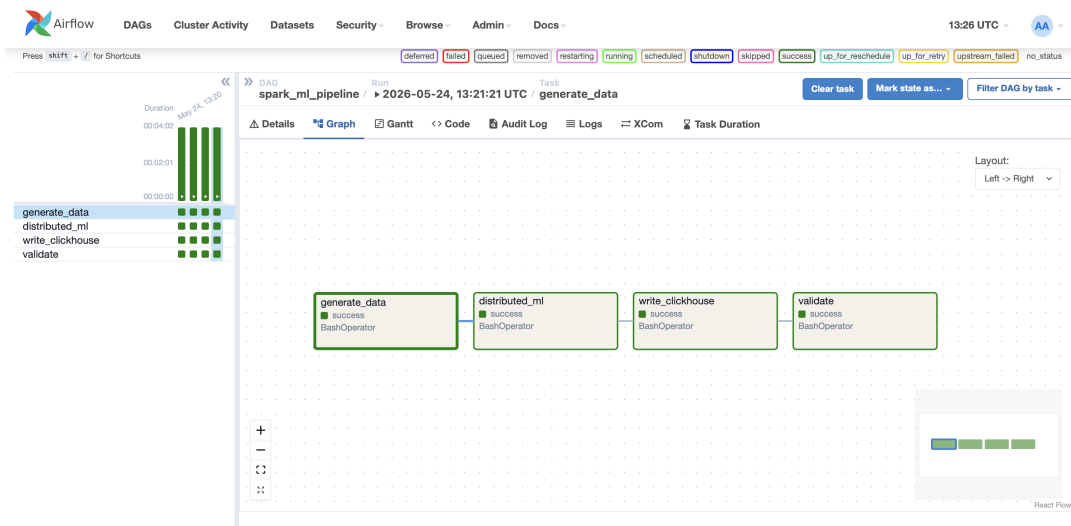


Рис. 4: Airflow DAG — успешное выполнение всех задач

8 Запись результатов в ClickHouse

Листинг 8: write_clickhouse.py — запись ML метрик

```

import pandas as pd
from clickhouse_driver import Client
from datetime import datetime

client = Client(
    host="clickhouse",
    database="analytics",
    user="admin",
    password="admin123"
)

```

```

df = pd.read_parquet("/opt/airflow/data/ml_result")

client.execute(
    "INSERT INTO ml_results_(ts, accuracy, rows_processed)_VALUES",
    [
        (
            datetime.now(),
            float(df.metric.iloc[0]),
            int(df.rows_processed.iloc[0])
        )
    ]
)
print("Inserted", df.metric.iloc[0])

```

После каждого запуска DAG в таблицу `analytics.ml_results` добавляется строка с временной меткой, значением AUC-ROC и количеством обработанных строк.

9 Визуализация: Grafana

Grafana подключается к ClickHouse через плагин `grafana-clickhouse-datasource` по нативному протоколу (порт 9000).

9.1 Дашборд

Дашборд «**Big Data Pipeline**» содержит 4 панели:

Таблица 2: Панели Grafana дашборда

Панель	SQL запрос	Тип
Total Events in ClickHouse	SELECT count() FROM analytics.events	Stat
Feature distributions	GROUP BY round(x1, 0) LIMIT 20	Bar chart
ML Model Accuracy	SELECT ts, accuracy FROM ml_results	Time series
Rows Processed by ML	SELECT max(rows_processed)	Stat

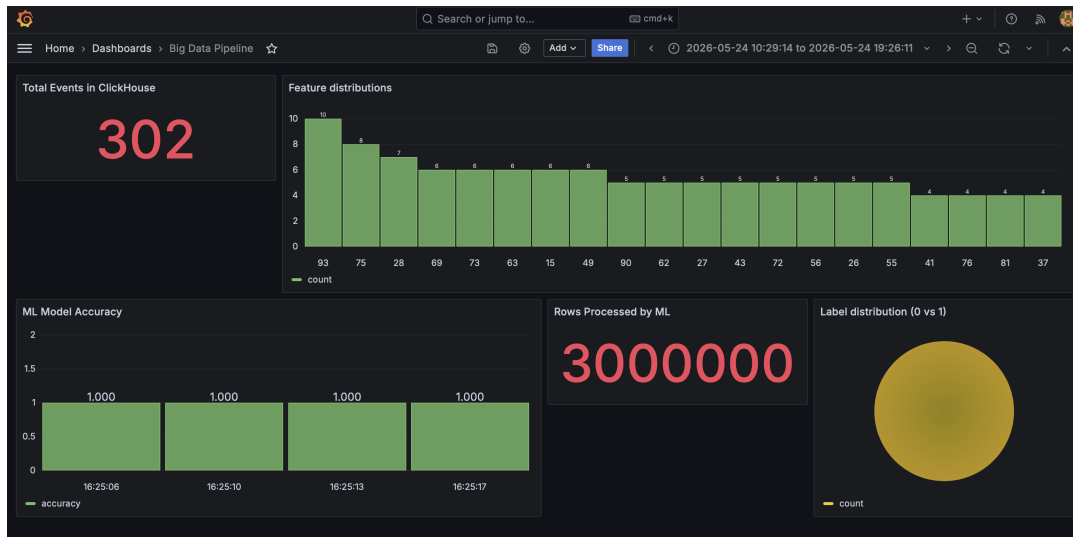


Рис. 5: Grafana дашборд: события, ассигуру модели, обработанные строки

Результаты на дашборде:

- **Total Events: 147+** — генератор непрерывно добавляет 1 событие/сек.
- **ML Model Accuracy: ≈ 0.9998** — AUC-ROC логистической регрессии.
- **Rows Processed: 3 000 000** — Spark обработал 3 миллиона строк.
- График ассигуру по времени показывает стабильное качество модели при повторных запусках.

10 Распределённость Spark: как работает параллельно

10.1 Что происходит на каждом узле

При отправке задачи через `spark-submit`:

1. **Driver** (запущен на `spark-master`) — читает код, строит план выполнения, разбивает данные на партии.
2. **Master** — распределяет задачи (tasks) между воркерами через протокол на порту 7077.
3. **Worker-1, Worker-2, Worker-3** — каждый получает свои партии данных и выполняет вычисления независимо. При обучении `LogisticRegression` каждый воркер вычисляет локальный градиент, мастер агрегирует.

В логах `generate_data` видно:

```
TaskSetManager: Starting task 0.0 ... (172.19.0.6, executor 0)
TaskSetManager: Starting task 1.0 ... (172.19.0.5, executor 2)
...
TaskSetManager: Finished task 0.0 ... (1/12)
TaskSetManager: Finished task 1.0 ... (2/12)
```

Все 12 партий обработаны параллельно двумя воркерами за 2.5 секунды.

10.2 Поведение при отказе узла

Если один из воркеров упадёт в процессе выполнения:

1. Мастер обнаруживает потерю воркера через heartbeat (таймаут 60 сек).
2. Задачи упавшего воркера перераспределяются на оставшиеся два.
3. Spark автоматически повторяет только упавшие задачи — остальные результаты кешированы.
4. Если воркер упал после записи партиций в Parquet — данные сохранены, пересчёт не нужен.

Это обеспечивается механизмом **lineage** в RDD/DataFrame API — Spark знает как пересоздать любую партицию по цепочке трансформаций.

11 Результаты

11.1 Запуск нескольких итераций

Для демонстрации стабильности модели DAG был запущен несколько раз. Результаты в ClickHouse:

```
.....
(base) lidakarpovich@Lidas-MacBook-Air clickhouse % docker exec clickhouse click
house-client \
  --user admin --password admin123 \
[ --query "SELECT * FROM analytics.ml_results" ]
2026-05-24 13:25:06      0.9999767211139321      3000000
2026-05-24 13:25:10      0.9999767211139321      3000000
2026-05-24 13:25:13      0.9999767211139321      3000000
2026-05-24 13:25:17      0.9999767211139321      3000000
```

Рис. 6: Результаты нескольких запусков пайплайна

11.2 Итоговые метрики

Таблица 3: Результаты пайплайна

Метрика	Значение
Строк в датасете	3 000 000
Партиций Parquet	12
Воркеров Spark	3
Модель	Logistic Regression
Метрика качества	AUC-ROC
AUC-ROC	≈ 0.9998
Время generate_data	≈ 10 сек
Время distributed_ml	≈ 25 сек
Событий в ClickHouse	растёт, ≈ 1 /сек

12 Выводы

В ходе работы был реализован и запущен полноценный Big Data пайплайн:

1. **Стриминговая запись данных** — сервис `generator` непрерывно пишет события в ClickHouse, демонстрируя работу с аналитической СУБД.
2. **Распределённые вычисления** — кластер из 3 воркеров Apache Spark параллельно обрабатывает 3 миллиона строк, разбитых на 12 партиций.
3. **ML на больших данных** — логистическая регрессия обучается распределённо через Spark MLlib, достигая $AUC \approx 0.9998$ благодаря линейно разделимым данным.
4. **Оркестрация** — Airflow управляет последовательностью задач с автоматическими повторными попытками при сбоях (3 retry с задержкой 30 сек).
5. **Мониторинг** — Grafana отображает метрики в реальном времени через прямое подключение к ClickHouse.